

DESARROLLO DE APLICACIONES EMPRESARIALES

Maestría en Software

Taller Práctico

Vibe Coding y Spec-Driven Development

De un ambiente de desarrollo conversacional con Kiro a una demo construida bajo especificación: el Cotizador de Reparación de Calzados

Duración estimada: 5 horas

Modalidad: En grupo de 4 integrantes

Contenido

1. Introducción y objetivos	5
1.1 Objetivo general	5
1.2 Objetivos específicos	5
1.3 Requisitos previos	5
1.4 Estructura y duración del taller	5
2. Sección 1 — Vibe Coding con Kiro: ambiente de desarrollo y pruebas	7
2.1 Objetivo de la sección	7
2.2 Contexto y alcance	7
2.3 Herramientas requeridas	7
2.4 Arquitectura objetivo del ambiente	7
2.5 Guía paso a paso	8
Paso 1 — Prompt de intención inicial	8
Paso 2 — Generación del docker-compose.yml	8
Paso 3 — Ajustes conversacionales	8
Paso 4 — Levantar y validar el ambiente	8
Paso 5 — Iteración adicional (bonus)	9
2.6 Banco de prompts sugeridos	9
2.7 Entregables de la sección	9
2.8 Preguntas de reflexión	9
2.9 Rúbrica de evaluación — Sección 1	9
3. Sección 2 — Spec-Driven Development con Kiro: Cotizador de Reparación de Calzados	11
3.1 Objetivo de la sección	11
3.2 Caso de negocio	11
3.3 Paso 1 — Especificación funcional	11
3.3.1 Historias de usuario	11
3.3.2 Reglas de negocio	11
3.3.3 Criterios de aceptación (Gherkin)	12
3.3.4 Especificación funcional de pantallas (frontend)	12
3.4 Paso 2 — Especificación técnica	13
3.4.1 Arquitectura hexagonal objetivo	13
3.4.2 Convenciones de nomenclatura	14
3.4.3 Patrones de diseño a aplicar	14
3.4.4 Modelo de dominio	15
3.4.5 Contrato de API (OpenAPI)	15
3.5 Paso 3 — Gate de revisión y aprobación de la especificación	16

3.6 Paso 4 — Cargar la especificación en Kiro y generar el código	16
3.6.1 Cómo organiza Kiro una especificación: Specs y Steering	16
3.6.2 Proyecto Backend - Cargar la especificación técnica como Steering	17
3.6.3 Crear el Feature Spec del backend y cargar la especificación funcional	17
3.6.4 Revisar y aprobar design.md y tasks.md	18
3.6.5 Ejecutar las tareas y validar contra la especificación	18
3.6.6 Proyecto Frontend - Preparar los steering files del proyecto frontend y repetir el flujo	19
3.6.6.1 architecture.md — arquitectura objetivo del frontend	19
3.6.6.2 conventions.md — convenciones de nomenclatura del frontend	20
3.6.6.3 design-patterns.md — patrones de diseño a aplicar en el frontend	20
3.6.7 Crear el Feature Spec del frontend y cargar la especificación funcional de pantallas	21
3.6.8 Ejecutar las tareas y validar contra la especificación	22
3.6.9 Estructura del proyecto backend	22
3.6.10 Estructura del proyecto frontend	23
3.7 Paso 5 — Opcional - Validación automatizada contra la especificación	23
3.8 Entregables de la sección	24
3.9 Rúbrica de evaluación — Sección 2	24
4. Sección 3 — Despliegue integrado con Vibe Coding y pruebas end-to-end	26
4.1 Objetivo de la sección	26
4.2 Contexto y alcance	26
4.3 Herramientas requeridas	26
4.4 Arquitectura objetivo del despliegue integrado	27
4.5 Guía paso a paso	27
Paso 1 — Prompt de intención inicial	27
Paso 2 — Generación del Dockerfile y del servicio backend	27
Paso 3 — Opcional - Ajustes conversacionales	27
Paso 4 — Levantar y validar el ambiente integrado	28
Paso 5 — Iteración adicional (bonus)	28
4.6 Pruebas integrales desde el navegador	28
4.6.1 Guion de pruebas manuales basado en los escenarios Gherkin	28
4.6.2 Opcional (bonus): automatizar las pruebas end-to-end con Playwright	29
4.7 Banco de prompts sugeridos	29
4.8 Entregables de la sección	30
4.9 Preguntas de reflexión	30
4.10 Rúbrica de evaluación — Sección 3	30
5. Cierre del taller: comparación de experiencia	32
5.1 Tabla de reflexión comparativa	32

5.2 Discusión final	32
Anexo A — docker-compose.yml de referencia	33
Anexo B — Contrato OpenAPI completo	34
Anexo C — Escenarios Gherkin completos	36
Anexo D — Fragmentos de código de referencia	37
Anexo E — Glosario	38
Anexo F — Artefactos de referencia para el despliegue integrado (Sección 3)	39

1. Introducción y objetivos

1.1 Objetivo general

Este taller pone en práctica, con dos ejercicios contrastantes, los dos paradigmas revisados en la sesión teórica: Vibe Coding y Spec-Driven Development. El propósito no es memorizar pasos, sino experimentar en carne propia las diferencias de velocidad, control, trazabilidad y riesgo entre ambos enfoques, para desarrollar criterio como futuro líder o arquitecto de software.

1.2 Objetivos específicos

- Desplegar un ambiente de desarrollo y pruebas con Docker, MySQL y Nginx usando Kiro en modo conversacional (vibe coding), documentando el proceso y sus riesgos.
- Redactar una especificación funcional y técnica completa para un caso de negocio simple (cotizador de reparación de calzados), incluyendo convenciones de nomenclatura, patrones de diseño y arquitectura hexagonal.
- Generar, a partir de esa especificación, dos proyectos simples (frontend y backend) usando Kiro en modo Spec-driven — es decir, restringido por requirements.md, design.md y tasks.md, no por la conversación libre.
- Comparar explícitamente la experiencia de ambos flujos: tiempo invertido, calidad percibida, trazabilidad y nivel de confianza en el resultado.
- Integrar, usando vibe coding con Kiro, los proyectos de backend y frontend de la Sección 2 dentro del ambiente Docker desplegado en la Sección 1, y validar el flujo completo de negocio con pruebas manuales desde el navegador.

1.3 Requisitos previos

- **Conocimientos:** fundamentos de Docker y contenedores, HTTP/REST, programación orientada a objetos, nociones de arquitectura hexagonal (puertos y adaptadores) vistas en clase.
- **Software instalado:** Docker Desktop (o Docker Engine + Compose plugin), Kiro como IDE agéntico con soporte de spec-driven development, un editor de código (VS Code recomendado), Node.js 18+, JDK 17+, Apache Maven 3.9+ para el stack sugerido en la Sección 2 y para empaquetar el backend en la Sección 3, cliente REST (Postman o curl) y un navegador con herramientas de desarrollador (DevTools) para las pruebas de la Sección 3.
- **Cuenta / acceso:** acceso a Kiro para las tres secciones (modo conversacional en las Secciones 1 y 3, modo Specs en la Sección 2).
- **Repositorio:** un repositorio Git vacío (local o en GitHub/GitLab) donde el estudiante registrará los tres entregables en carpetas separadas: /seccion-1-vibe-coding, /seccion-2-spec-driven y /seccion-3-despliegue-integrado.

1.4 Estructura y duración del taller

Bloque	Contenido	Duración estimada
Sección 1	Vibe Coding con Kiro: despliegue de ambiente Docker + MySQL + Nginx	1 hora
Sección 2	Spec-Driven Development: cotizador de reparación de calzados (frontend + backend)	2 - 3 horas
Sección 3	Vibe Coding con Kiro: integración del backend y el frontend sobre el ambiente Docker de la Sección 1, con pruebas end-to-end desde el navegador	1 hora
Cierre	Comparación de experiencia y discusión grupal	20 - 30 minutos

Nota metodológica

Las tres secciones se evalúan con criterios distintos (ver rúbricas 2.9, 3.9 y 4.10). La Sección 1 premia agilidad y funcionalidad; la Sección 2 premia trazabilidad, apego a la especificación y calidad arquitectónica; la Sección 3 vuelve a premiar agilidad y funcionalidad, como la Sección 1, pero exige además que la integración quede evidenciada con pruebas reales, no solo declarada. Esa asimetría es intencional: refleja el criterio real que un líder técnico debe aplicar según el contexto.

2. Sección 1 — Vibe Coding con Kiro: ambiente de desarrollo y pruebas

2.1 Objetivo de la sección

Desplegar, mediante conversación directa con Kiro y sin redactar una especificación previa, un ambiente local de desarrollo y pruebas compuesto por un contenedor de base de datos MySQL y un contenedor Nginx, orquestados con Docker Compose. El énfasis está en la velocidad de iteración conversacional: se detona el resultado a partir de una intención en lenguaje natural y se corrige sobre la marcha.

2.2 Contexto y alcance

Este es exactamente el tipo de tarea señalada en la matriz de decisión de la sesión teórica como apta para vibe coding: infraestructura de desarrollo/pruebas (no de producción), de bajo riesgo, reversible en segundos (basta con “docker compose down”) y sin integraciones críticas de terceros. No se espera una especificación formal; se espera fluidez conversacional y buen criterio para saber cuándo el resultado “ya sirve”.

Límite del ejercicio

Este ambiente es exclusivamente para desarrollo y pruebas locales. No debe usarse como referencia para un despliegue en producción: no incluye TLS real, gestión de secretos, alta disponibilidad ni hardening — precisamente los elementos que exigirían pasar a un enfoque spec-driven, como se discute en la Sección 3.9 y en el cierre del taller.

2.3 Herramientas requeridas

- Kiro como IDE agéntico en modo conversacional / chat.
- Docker Desktop con Docker Compose v2.
- Un cliente MySQL (línea de comandos, MySQL Workbench o DBeaver) para validar la conexión.
- Navegador web para validar Nginx.

2.4 Arquitectura objetivo del ambiente

El resultado esperado de la conversación con Kiro debe aproximarse al siguiente conjunto de servicios, aunque los estudiantes tienen libertad de nombrar contenedores, puertos y variables a su criterio — esa es precisamente la naturaleza exploratoria del vibe coding.

Servicio	Imagen base sugerida	Propósito	Puerto host
db	mysql:8.0	Base de datos relacional para desarrollo y pruebas	3306
web	nginx:alpine	Servidor web / proxy inverso hacia una futura app	8080

Se espera además: una red Docker dedicada, al menos un volumen nombrado para persistir los datos de MySQL entre reinicios, variables de entorno para credenciales (no hardcodeadas en la imagen) y un archivo de configuración básico de Nginx servido como estático o como proxy.

2.5 Guía paso a paso

Paso 1 — Prompt de intención inicial

Abrir Kiro sobre una carpeta de proyecto vacía (ej. seccion-1-vibe-coding/) y describir la intención completa en una sola conversación, sin detallar sintaxis de Docker Compose. El objetivo es practicar la fluidez conversacional propia del vibe coding.

Prompt sugerido (adaptar libremente):

```
Necesito un ambiente local de desarrollo y pruebas con Docker Compose.
Quiero un contenedor de MySQL 8 con una base de datos llamada
"tallerdae", usuario y contraseña de desarrollo, y persistencia de datos
con un volumen. También quiero un contenedor de Nginx que sirva una
página estática simple de bienvenida en el puerto 8080, y que en el futuro
pueda actuar como proxy hacia un backend. Todo debe poder levantarse con
un solo comando y debe incluir un archivo .env para las credenciales.
```

Paso 2 — Generación del docker-compose.yml

Dejar que Kiro proponga la estructura completa (docker-compose.yml, .env, carpeta nginx/ con su configuración y contenido estático). Revisar el resultado ejecutando el ambiente, sin todavía leer línea por línea cada archivo generado — coherente con la naturaleza del vibe coding.

Paso 3 — Ajustes conversacionales

Iterar en lenguaje natural sobre el resultado inicial. Ejemplos de instrucciones de seguimiento que se recomienda practicar:

- “Agrega un healthcheck a MySQL para que Nginx no arranque hasta que la base de datos esté lista.”
- “Mueve las credenciales de MySQL a un archivo .env y no las dejes escritas directamente en el docker-compose.yml.”
- “Expón MySQL únicamente dentro de la red interna de Docker, no lo publiques en el host.” (y luego discutir el trade-off con la clase)
- “Agrega un volumen para los logs de Nginx que pueda revisar desde mi máquina.”

Paso 4 — Levantar y validar el ambiente

Comandos de validación:

```
docker compose up -d
docker compose ps
docker compose logs db --tail=50
curl -I http://localhost:8080
mysql -h 127.0.0.1 -P 3306 -u dev_user -p tallerdae
```

Registrar evidencia (capturas de pantalla o salida de consola) de que ambos servicios están arriba (“Up” / “healthy”) y de una conexión exitosa a la base de datos.

Paso 5 — Iteración adicional (bonus)

Con el mismo estilo conversacional, pedir a Kiro al menos una mejora adicional a elección, por ejemplo:

- Agregar Adminer o phpMyAdmin para administrar MySQL desde el navegador.
- Configurar Nginx como proxy inverso hacia un contenedor de aplicación de prueba (ej. una imagen “hello-world” en Node o Python).
- Agregar un script setup.sh que automatice el primer arranque (crear .env desde una plantilla, ejecutar docker compose up, esperar el healthcheck).

2.6 Banco de prompts sugeridos

Prompts adicionales que pueden usar como inspiración si se quedan sin ideas para iterar; no es necesario usarlos todos.

1. “Explicame en dos frases qué hace cada servicio de este docker-compose.yml, como si yo no lo hubiera pedido.”
2. “Simula que este ambiente falla al arrancar porque el puerto 3306 ya está ocupado; dime cómo lo solucionarías.”
3. “Agrega un archivo README.md que explique cómo levantar y apagar el ambiente.”
4. “Sugiere qué cosas de este ambiente NO deberían usarse tal cual en producción.”

2.7 Entregables de la sección

- Carpeta seccion-1-vibe-coding/ con docker-compose.yml, .env.example, configuración de Nginx y assets estáticos.
- Captura o log de docker compose ps mostrando ambos servicios corriendo.
- Captura o log de una conexión exitosa a MySQL.
- Bitácora breve (mín. 10 líneas) de la conversación con Kiro: qué se pidió, qué se ajustó y por qué.
- Respuestas a las preguntas de reflexión (sección 2.8).

2.8 Preguntas de reflexión

1. ¿Cuánto tiempo tomó desde el primer prompt hasta tener el ambiente funcionando? ¿Cuántas iteraciones conversacionales fueron necesarias?
2. ¿Revisaste línea por línea el docker-compose.yml generado, o confiaste en que “funcionaba”? ¿Qué riesgo implica esa decisión?
3. Si este mismo ambiente tuviera que promoverse a un entorno de staging compartido por todo el equipo, ¿qué cambiaría en tu forma de trabajar? Relaciona tu respuesta con la matriz de decisión de la sesión teórica.
4. ¿Qué credenciales o configuraciones sensibles quedaron expuestas durante el ejercicio? ¿Cómo las gobernarías en un contexto empresarial real?

2.9 Rúbrica de evaluación — Sección 1

Criterio	Insuficiente (0-59)	Aceptable (60-79)	Sobresaliente (80-100)
Funcionalidad	El ambiente no levanta o falta un servicio	Ambos servicios levantan, con ajustes manuales	Levanta con un solo comando, con healthchecks
Persistencia y config.	Sin volúmenes ni variables de entorno	Volumen o .env presentes, no ambos	Volumen, .env y red dedicada correctamente usados
Documentación del proceso	Sin bitácora de prompts	Bitácora breve y genérica	Bitácora clara que muestra iteración real
Reflexión crítica	Respuestas superficiales	Respuestas correctas pero generales	Conecta explícitamente con riesgos y matriz de decisión

3. Sección 2 — Spec-Driven Development con Kiro: Cotizador de Reparación de Calzados

3.1 Objetivo de la sección

Construir, usando Kiro y siguiendo estrictamente su flujo nativo de especificaciones (requirements.md → design.md → tasks.md, con aprobación humana entre cada fase), una demo compuesta por un proyecto de backend y un proyecto de frontend muy simples que resuelvan un cotizador de reparación de calzados, aplicando arquitectura hexagonal, convenciones de nomenclatura y patrones de diseño explícitos. A diferencia de la Sección 1, aquí Kiro no se usa en modo conversacional libre: se usa en modo “Spec”, donde el propio Kiro obliga a pasar por fases de revisión antes de generar código.

3.2 Caso de negocio

Un taller artesanal de reparación de calzado quiere ofrecer a sus clientes una cotización estimada en línea antes de que traigan el calzado físicamente. El cliente selecciona el tipo de calzado y una o más reparaciones deseadas, indica si el servicio es urgente, y el sistema calcula el total estimado y el tiempo de entrega.

Alcance intencionalmente acotado

No hay autenticación, sin base de datos no hay persistencia real obligatoria (un repositorio en memoria es suficiente) y no hay pasarela de pago. El foco pedagógico es la disciplina de especificación y la arquitectura, no la cobertura funcional del negocio.

3.3 Paso 1 — Especificación funcional

3.3.1 Historias de usuario

- **HU-01:** Como cliente, quiero seleccionar un tipo de calzado y una o más reparaciones para obtener una cotización estimada del costo total.
- **HU-02:** Como cliente, quiero marcar el servicio como urgente para conocer el recargo aplicable y el nuevo tiempo estimado de entrega.
- **HU-03:** Como cliente, quiero consultar los tipos de calzado y reparaciones disponibles antes de generar la cotización, para saber qué puedo seleccionar.

3.3.2 Reglas de negocio

- **RN-01:** El subtotal de la cotización es la suma, para cada reparación seleccionada, de (precio base de la reparación × factor de complejidad del tipo de calzado).
- **RN-02:** Si el servicio es urgente, se aplica un recargo del 30% sobre el subtotal. El total = subtotal + recargo (si aplica).
- **RN-03:** El tiempo estimado de entrega es el máximo entre los tiempos estimados (en días) de las reparaciones seleccionadas. Si el servicio es urgente, ese tiempo se reduce a la mitad, redondeado hacia arriba, con un mínimo de 1 día.
- **RN-04:** Una cotización debe tener al menos una reparación seleccionada; de lo contrario la solicitud se rechaza.

- **RN-05:** Toda cotización generada debe recibir un identificador único y una fecha de creación.

3.3.3 Criterios de aceptación (Gherkin)

Estos escenarios son el contrato de comportamiento contra el cual se validará el backend en el Paso 5 (sección 3.7). El listado completo, con más casos borde, está en el Anexo C.

Característica: Generar cotización de reparación de calzado

Escenario: Cotización simple sin urgencia

Dado un calzado "Zapato formal" con factor de complejidad 1.2

Y una reparación "Cambio de tacón" con precio base 12.00 y tiempo estimado 2 días

Cuando el cliente solicita una cotización sin marcar el servicio como urgente

Entonces el total de la cotización debe ser 14.40

Y el tiempo estimado de entrega debe ser 2 días

Escenario: Cotización urgente con recargo

Dado un calzado "Bota de cuero" con factor de complejidad 1.5

Y una reparación "Cambio de suela" con precio base 20.00 y tiempo estimado 4 días

Cuando el cliente solicita una cotización marcando el servicio como urgente

Entonces el subtotal debe ser 30.00

Y el total debe ser 39.00

Y el tiempo estimado de entrega debe ser 2 días

Escenario: Cotización sin reparaciones seleccionadas

Cuando el cliente solicita una cotización sin seleccionar ninguna reparación

Entonces el sistema debe rechazar la solicitud con un error de validación

3.3.4 Especificación funcional de pantallas (frontend)

Las historias de usuario y reglas de negocio anteriores describen qué calcula el sistema, pero no cómo se ve ni cómo se comporta la pantalla que lo expone. Esta subsección es la que se cargará en el Feature Spec del frontend (sección 3.6.7): define la pantalla única del cotizador, sus componentes, sus estados y las reglas de interacción que Kiro debe respetar al diseñar la interfaz.

- **Pantalla única:** "Cotizador de reparación". No hay navegación entre pantallas ni rutas; todo ocurre en index.html.

Componentes de la pantalla:

Componente	Tipo de control	Origen de los datos
Selector de tipo de calzado	Lista desplegable (select)	GET /api/tipos-calzado
Lista de reparaciones	Casillas de verificación (checkbox), una por tipo de reparación	GET /api/tipos-reparacion
Servicio urgente	Interruptor o casilla de verificación única	Estado local del formulario
Botón "Cotizar"	Botón de envío	Dispara POST /api/cotizaciones
Panel de resultado	Bloque de texto (subtotal, recargo, total, tiempo estimado)	Respuesta de POST /api/cotizaciones
Mensaje de error	Bloque de alerta en línea, sin modal	Errores de validación (RN-04) o de red/servidor

Estados de la pantalla:

- **UI-E1 Cargando catálogo:** al abrir la pantalla, mientras se resuelven las dos peticiones GET, el selector y las casillas se muestran deshabilitados con un indicador de carga.
- **UI-E2 Lista para cotizar:** catálogo cargado; el usuario puede seleccionar tipo de calzado, reparaciones y urgencia.
- **UI-E3 Cotizando:** tras pulsar “Cotizar”, el botón queda deshabilitado y muestra un indicador de carga hasta recibir respuesta.
- **UI-E4 Resultado mostrado:** el panel de resultado se llena con la respuesta del backend (subtotal, recargo, total, tiempo estimado).
- **UI-E5 Error:** se muestra el mensaje de error correspondiente (validación o servidor) sin perder la selección hecha por el usuario.

Reglas de interacción:

- **UI-01:** El botón “Cotizar” permanece deshabilitado hasta que haya un tipo de calzado seleccionado y al menos una reparación marcada (refleja la RN-04 del backend en la interfaz, para no depender solo del error del servidor).
- **UI-02:** Mientras el estado es UI-E3 (cotizando), el botón no puede volver a pulsarse (evita cotizaciones duplicadas por doble clic).
- **UI-03:** Si la respuesta del backend es un error 400, el mensaje mostrado debe ser el que devuelve la API, no un texto genérico, y la selección del usuario permanece intacta para que pueda corregirla.
- **UI-04:** Si el usuario modifica cualquier selección después de ver un resultado (UI-E4), el panel de resultado se oculta hasta que se vuelva a pulsar “Cotizar”, para no mostrar un total que ya no corresponde a la selección visible.

Criterios de aceptación de interfaz (Gherkin). El listado completo, con más casos borde, está en el Anexo C:

Característica: Interacción de la pantalla del cotizador

Escenario: Botón deshabilitado sin selección completa

Dado que el catálogo de calzados y reparaciones ya se cargó

Y el cliente no ha seleccionado ninguna reparación

Entonces el botón "Cotizar" debe estar deshabilitado

Escenario: Mostrar resultado tras cotizar exitosamente

Dado que el cliente seleccionó un calzado y al menos una reparación

Cuando el cliente presiona "Cotizar"

Entonces el panel de resultado debe mostrar el subtotal, el total

y el tiempo estimado de entrega devueltos por la API

Escenario: Ocultar el resultado anterior al cambiar la selección

Dado que la pantalla ya muestra un resultado de una cotización previa

Cuando el cliente cambia el tipo de calzado seleccionado

Entonces el panel de resultado debe ocultarse hasta la siguiente cotización

3.4 Paso 2 — Especificación técnica

3.4.1 Arquitectura hexagonal objetivo

El backend debe organizarse en tres capas concéntricas, con dependencias apuntando siempre hacia el dominio (nunca al revés):

- **domain:** entidades, objetos de valor y excepciones de negocio. Sin dependencias a frameworks ni a infraestructura.
- **application:** puertos de entrada (casos de uso), puertos de salida (contratos de repositorio) y los servicios que implementan los casos de uso orquestando el dominio.
- **infrastructure:** adaptadores de entrada (controladores REST, DTOs, mappers) y adaptadores de salida (repositorios en memoria o JPA, configuración del framework).

Representación textual de las dependencias permitidas:

```
infrastructure.adapter.in.rest ---> application.port.in ---> domain
infrastructure.adapter.out.persistence ---> application.port.out ---> domain
application.service (implementa port.in, usa port.out) ---> domain
```

3.4.2 Convenciones de nomenclatura

Elemento	Convención	Ejemplo
Paquete raíz	minúsculas, invertido de dominio	com.tallerdae.cotizador
Entidad / Value Object de dominio	PascalCase, sustantivo, sin sufijo técnico	Cotizacion, Calzado, Dinero
Puerto de entrada (caso de uso)	PascalCase + sufijo UseCase	GenerarCotizacionUseCase
Puerto de salida (repositorio)	PascalCase + sufijo RepositoryPort	CotizacionRepositoryPort
Implementación de caso de uso	PascalCase + sufijo Service	GenerarCotizacionService
Adaptador REST de entrada	PascalCase + sufijo Controller	CotizacionController
Adaptador de persistencia	PascalCase + sufijo (InMemory Jpa)Adapter	InMemoryCotizacionRepositoryAdapter
DTO de request/response	PascalCase + sufijo Request / Response	CotizacionRequest, CotizacionResponse
Mapper dominio $\rightleftharpoons$ DTO	PascalCase + sufijo Mapper	CotizacionMapper
Método	camelCase, verbo que expresa intención	calcularTotal(), generarCotizacion()
Constante	MAYÚSCULAS_CON_GUION_BAJO	RECARGO_URGENCIA_PORCENTAJE

3.4.3 Patrones de diseño a aplicar

Patrón	Dónde se aplica	Justificación
Strategy	Cálculo del recargo por nivel de urgencia (UrgencyPricingStrategy)	Permite agregar nuevos niveles de urgencia sin modificar la lógica existente (abierto/cerrado)

Patrón	Dónde se aplica	Justificación
Factory Method	Creación de Cotizacion mediante un método estático de fábrica que valida invariantes (RN-04, RN-05)	Garantiza que no exista una Cotizacion en estado inválido
Repository	CotizacionRepositoryPort, CalzadoRepositoryPort, ReparacionRepositoryPort	Desacopla el dominio de la tecnología de persistencia concreta
DTO + Mapper	CotizacionRequest/Response y CotizacionMapper	Evita que el modelo de dominio quede acoplado al contrato HTTP
Inyección de dependencias	Construcción de servicios de aplicación con sus puertos de salida	Invierte el control entre application e infrastructure (regla de dependencia hexagonal)

3.4.4 Modelo de dominio

Entidades y objetos de valor mínimos requeridos, con sus atributos principales:

Tipo	Nombre	Atributos principales
Entidad	Calzado	id, nombre, factorComplejidad
Entidad	TipoReparacion	id, nombre, precioBase, tiempoEstimadoDias
Agregado raíz	Cotizacion	id, calzado, reparaciones, urgente, subtotal, total, tiempoEstimadoDias, fechaCreacion
Value Object	Dinero	monto, moneda (operaciones: sumar, aplicarPorcentaje)
Enum	NivelUrgencia	NORMAL, URGENTE

3.4.5 Contrato de API (OpenAPI)

Fragmento del contrato; la versión completa está en el Anexo B. Este contrato es la fuente de verdad tanto para el backend como para el frontend — ningún endpoint debe implementarse sin aparecer aquí primero.

```
paths:
  /api/tipos-calzado:
    get:
      summary: Lista los tipos de calzado disponibles
      responses:
        '200':
          description: Listado de tipos de calzado
  /api/tipos-reparacion:
    get:
      summary: Lista los tipos de reparación disponibles
      responses:
        '200':
          description: Listado de tipos de reparación
  /api/cotizaciones:
    post:
      summary: Genera una cotización estimada
      requestBody:
        content:
          application/json:
            schema:
```

```

    type: object
    required: [tipoCalzadoId, tipoReparacionIds]
    properties:
      tipoCalzadoId: { type: string }
      tipoReparacionIds:
        type: array
        minItems: 1
        items: { type: string }
      urgente: { type: boolean, default: false }
  responses:
    '201':
      description: Cotización generada
    '400':
      description: Solicitud inválida (p. ej. sin reparaciones)

```

3.5 Paso 3 — Gate de revisión y aprobación de la especificación

Antes de pedirle a Kiro que genere una sola línea de código, el estudiante debe autoevaluar la especificación producida en los pasos anteriores con la siguiente checklist, actuando como su propio comité de arquitectura. Solo si todos los puntos están marcados se avanza al Paso 4.

- Cada endpoint del contrato de API tiene al menos un escenario Gherkin que lo cubre.
- Todas las reglas de negocio (RN-01 a RN-05) están reflejadas en algún escenario de aceptación.
- El modelo de dominio no tiene atributos ni entidades no mencionados en las historias de usuario o reglas de negocio (evitar sobre-especificar).
- Las convenciones de nomenclatura son consistentes entre sí (por ejemplo, no mezclar RepositoryPort con Repository a secas).
- Cada patrón de diseño listado tiene un punto de aplicación concreto y justificado, no es un adorno teórico.
- Cada estado de pantalla (UI-E1 a UI-E5) y cada regla de interacción (UI-01 a UI-04) de la sección 3.3.4 tiene al menos un escenario Gherkin que lo cubre.

Por qué este paso no es opcional — y cómo se relaciona con Kiro

Este gate manual es la preparación para los gates que el propio Kiro impondrá automáticamente en el Paso 4: Kiro nunca pasa de requirements.md a design.md, ni de design.md a tasks.md, sin que el estudiante apruebe explícitamente cada documento. Si la especificación que se carga aquí ya está completa y validada, esas aprobaciones dentro de Kiro serán rápidas; si está incompleta, Kiro las reflejará con huecos o supuestos incorrectos en sus tres documentos, y el estudiante los detectará recién en el Paso 4 — con más costo de retrabajo que si se detectan ahora.

3.6 Paso 4 — Cargar la especificación en Kiro y generar el código

A partir de este punto, toda interacción con Kiro debe ocurrir dentro de su flujo nativo de especificaciones (“Specs”), no en el chat conversacional libre que se usó en la Sección 1. Esta sección explica, mecánicamente, cómo entra la especificación de las secciones 3.3 y 3.4 a Kiro y cómo Kiro la convierte en código.

3.6.1 Cómo organiza Kiro una especificación: Specs y Steering

Kiro trabaja con dos tipos de artefactos persistidos como archivos Markdown dentro del repositorio, no solo como mensajes de chat que se pierden al cerrar la conversación:

- **Steering files** (carpeta `.kiro/steering/`): contexto que aplica a todo el proyecto, en todas las tareas y specs. Aquí es donde vive la especificación técnica transversal: arquitectura hexagonal, convenciones de nomenclatura y patrones de diseño (secciones 3.4.1 a 3.4.3).
- **Feature Specs** (carpeta `.kiro/specs/<nombre-feature>/`): contexto específico de una funcionalidad, compuesto por tres archivos que Kiro genera en orden y que el estudiante revisa y aprueba uno por uno: `requirements.md`, `design.md` y `tasks.md`. Aquí es donde vive la especificación funcional del cotizador (sección 3.3) y el contrato de API (3.4.5).

Regla práctica para decidir dónde va cada cosa

Si una regla aplica a todo lo que Kiro construya en este repositorio (convenciones de nombres, capas hexagonales, patrones), va en un steering file. Si una regla aplica solo al cotizador como funcionalidad (historias de usuario, reglas de negocio, criterios de aceptación, endpoints), va en el Feature Spec. Cargar todo como un solo bloque de texto en el chat, sin esta separación, es la forma más común de que Kiro pierda de vista las convenciones a mitad de la implementación.

3.6.2 Proyecto Backend - Cargar la especificación técnica como Steering

Antes de crear ningún Feature Spec, preparar el contexto persistente del proyecto backend (Cierre el proyecto o carpeta actual y abra nueva carpeta **seccion-2-spec-driven-back**):

1. En el panel de Kiro, ir a la sección Steering y crear un archivo nuevo llamado `architecture.md`.
2. Pegar en ese archivo el contenido de la sección 3.4.1 (capas hexagonales y reglas de dependencia) tal como está redactado en este documento.
3. Crear un segundo archivo `conventions.md` con la tabla completa de la sección 3.4.2.
4. Crear un tercer archivo `design-patterns.md` con la tabla de la sección 3.4.3, incluyendo la columna “Justificación” — Kiro la usa para decidir cuándo aplicar cada patrón, no solo qué patrón existe.
5. Usar el botón Refine sobre cada archivo: Kiro reformatea el texto pegado a su propio estilo de steering sin cambiar el contenido técnico.

Estos tres archivos quedan disponibles automáticamente en todas las tareas que Kiro ejecute en este proyecto, sin que el estudiante tenga que volver a pegarlos en cada prompt.

3.6.3 Crear el Feature Spec del backend y cargar la especificación funcional

1. Abrir el panel de Specs de Kiro y crear un nuevo Feature Spec. Cuando Kiro pregunte el flujo de trabajo, elegir Requirements-First (se parte de requisitos, no de un diseño técnico ya decidido).
2. En el cuadro de descripción inicial, no escribir una frase corta: pegar el contenido completo de las historias de usuario (3.3.1) y las reglas de negocio (3.3.2), y agregar al final una instrucción explícita de alcance.

Descripción inicial sugerida para el Feature Spec (pegar historias de usuario y reglas de negocio antes de esta instrucción):

Con base en las historias de usuario y reglas de negocio anteriores, genera el `requirements.md` de un backend de cotizaciones de reparación de calzado. Usa notación EARS para cada criterio de aceptación. No agregues funcionalidades no mencionadas arriba (sin autenticación, sin pagos, sin persistencia real: un repositorio en memoria es suficiente).

3. Kiro genera `requirements.md`. Compararlo línea por línea contra los escenarios Gherkin de la sección 3.3.3: cada escenario debe reconocerse en algún criterio de aceptación EARS - *Easy Approach to Requirements*

Syntax (“CUANDO... EL SISTEMA DEBERÁ...”). Si algo no coincide, no se aprueba todavía: se edita `requirements.md` directamente o se le pide a Kiro un ajuste puntual en el chat.

4. Solo cuando `requirements.md` refleje fielmente las secciones 3.3.1-3.3.3, aprobar para que Kiro pase a la fase de diseño.

3.6.4 Revisar y aprobar `design.md` y `tasks.md`

Kiro genera `design.md` combinando `requirements.md` recién aprobado con los tres steering files del paso 3.6.2. Validar contra la especificación técnica antes de aprobar:

- El diagrama o la descripción de componentes en `design.md` respeta las tres capas de la sección 3.4.1 (domain, application, infrastructure), sin mezclar responsabilidades.
- Las clases y puertos que Kiro propone en `design.md` usan los sufijos de la tabla de convenciones (UseCase, RepositoryPort, Service, Controller, Request/Response, Mapper).
- Los cinco patrones de diseño de la sección 3.4.3 aparecen mencionados en `design.md` con el mismo punto de aplicación acordado (por ejemplo, Strategy en el cálculo del recargo por urgencia, no en otro lugar).

Si `design.md` se desvía de la especificación técnica, no se aprueba: se corrige directamente en el archivo o se le indica a Kiro el ajuste, citando la sección del documento que se está incumpliendo (por ejemplo: “Ajusta el diseño: el repositorio debe ser un puerto de salida según la sección 3.4.1, no una dependencia directa del servicio”).

Una vez aprobado `design.md`, Kiro genera `tasks.md`: una lista de tareas discretas y ordenadas por dependencias. Revisar que:

- Cada tarea sea lo suficientemente pequeña como para completarse y revisarse de una sola vez (idealmente, una clase o un puerto por tarea).
- El orden de tareas no contradiga la arquitectura hexagonal (por ejemplo, que domain no dependa de tareas de infrastructure).

Aprobar `tasks.md` únicamente cuando ambos puntos se cumplan.

3.6.5 Ejecutar las tareas y validar contra la especificación

Con `tasks.md` aprobado, ejecutar las tareas una por una — nunca con la opción de “ejecutar todas las tareas” de una vez, incluso si Kiro la ofrece: el valor pedagógico de esta sección está en revisar cada pieza generada contra la spec, no en la velocidad. En la sesión activa de Kiro dar la siguiente instrucción:

Ejecuta las tareas en orden una por una. Pide mi aprobación antes de ejecutar cada tarea

1. Cuando Kiro termine, abrir el archivo generado y verificar: ¿el nombre de la clase o método coincide con la tabla de convenciones? ¿la clase está en el paquete que le corresponde según la arquitectura hexagonal?
2. Si Kiro pide permiso para instalar dependencias o ejecutar comandos, leer qué va a instalar antes de aceptar — igual que en la Sección 1, aceptar sin leer reproduce el riesgo del vibe coding dentro de un flujo que se suponía disciplinado.
3. Verificar la ejecución de cada tarea en el archivo `tasks.md`.

4. En este laboratorio no ejecute las tareas relacionadas a pruebas ya que puede consumir mas tokens y hacerle falta para culminar el taller.

Durante la ejecución, si es necesario recordarle a Kiro un artefacto puntual de la especificación puede usar el proveedor de contexto #spec para incluir automáticamente los tres archivos del Feature Spec en el mensaje, en lugar de volver a pegarlos a mano.

Al final de la ejecución de tareas revisar si se encuentra el archivo openapi.yaml. Si no es así, solicite la creación:

Genera el contrato OpenAPI para el servicio REST Cotizador de Reparación de Calzado que expone los tres grupos de endpoints mencionados en el design.md

Verifique que el contrato de servicio expone los endpoints y estructura definidos en el Anexo B. Pueden ser similares en forma pero en definición debe ser exacto.

Finalmente, empaquetes el proyecto en archivo .jar para que quede disponible para el despliegue posterior:

Compila el proyecto java cotizador y genera el .jar para desplegarlo en un ambiente docker

Verifique los artefactos creados: cotizador-0.0.1-SNAPSHOT.jar y Dockerfile

3.6.6 Proyecto Frontend - Preparar los steering files del proyecto frontend y repetir el flujo

El frontend se implementa como un Feature Spec independiente (por ejemplo, cotizador-frontend), en su propio workspace de Kiro, con sus propios steering files. No tiene sentido copiar tal cual los steering files del backend: la arquitectura hexagonal, el patrón Repository o el patrón Strategy existen para proteger reglas de negocio complejas, y en este frontend deliberadamente simple (HTML + CSS + JavaScript sin framework) no hay reglas de negocio que proteger — esas ya viven en el backend. Cargar en Kiro una especificación técnica de ese calibre para un frontend tan pequeño produciría exactamente el problema opuesto al que se busca evitar: sobre-especificación.

Antes de crear ningún Feature Spec, preparar el contexto persistente del proyecto frontend (Cierre el proyecto o carpeta actual y abra nueva carpeta **seccion-2-spec-driven-front**)

Principio para adaptar la especificación técnica al stack

La especificación técnica no se copia entre proyectos: se adapta a lo que el stack necesita para mantenerse ordenado. Un backend con reglas de negocio pide capas y puertos; un frontend sin framework que solo pinta un formulario y llama tres endpoints pide, como mucho, separar “qué se ve” de “qué se sabe” de “cómo se habla con el servidor”.

3.6.6.1 architecture.md — arquitectura objetivo del frontend

En lugar de las tres capas hexagonales del backend, el frontend usa una separación mucho más liviana en tres responsabilidades dentro de la carpeta js/, pensada para JavaScript sin framework y sin bundler:

Archivo	Responsabilidad	Puede depender de
index.html + css/estilos.css	Estructura y estilo visual. Expone elementos con id estables para que app.js los enlace. No contiene lógica.	Nada

Archivo	Responsabilidad	Puede depender de
js/state.js	Mantiene en memoria el estado de la aplicación (catálogo cargado, selección actual del usuario, última cotización recibida).	Nada — no toca el DOM ni hace fetch
js/api.js	Único módulo que conoce las URLs del backend; hace fetch y traduce las respuestas HTTP a objetos JavaScript simples.	Nada — no conoce el DOM ni el estado
js/app.js	Escucha eventos del DOM, coordina state.js y api.js, y decide qué volver a pintar en pantalla.	state.js y api.js

Regla de dependencia única (equivalente liviano de la regla hexagonal del backend): state.js y api.js no se conocen entre sí ni conocen el DOM; toda coordinación pasa por app.js. Esto evita, por ejemplo, que una función de renderizado termine haciendo fetch directamente, o que api.js manipule elementos del HTML.

3.6.6.2 conventions.md — convenciones de nomenclatura del frontend

Elemento	Convención	Ejemplo
Archivos JavaScript	camelCase, un sustantivo que describe su responsabilidad	api.js, app.js, state.js
IDs de elementos HTML	kebab-case, descriptivo del control	tipo-calzado-select, boton-cotizar, resultado-cotizacion
Clases CSS	kebab-case, BEM ligero (bloque__elemento--modificador) solo donde aporte claridad	cotizador__resultado, cotizador__resultado--urgente
Funciones	camelCase, verbo que expresa intención	obtenerTiposCalzado(), renderizarResultado(), construirRequestCotizacion()
Variables de estado	camelCase, sustantivo	cotizacionActual, tiposCalzadoDisponibles, servicioEsUrgente
Constantes de configuración	MAYÚSCULAS_CON_GUION_BAJO	API_BASE_URL
Eventos personalizados (si se usan)	kebab-case con prefijo del módulo de origen	cotizador:resultado-listo

3.6.6.3 design-patterns.md — patrones de diseño a aplicar en el frontend

Se sugieren cuatro patrones livianos, todos expresables en JavaScript nativo sin librerías. Los cinco patrones del backend (Strategy, Factory Method, Repository, DTO+Mapper, inyección de dependencias) no se piden aquí: el frontend no tiene reglas de negocio propias ni múltiples implementaciones intercambiables que justifiquen esa maquinaria.

Patrón	Dónde se aplica	Justificación
Module pattern (ES Modules)	api.js y state.js exportan únicamente funciones públicas con export; el resto de su contenido queda privado al archivo	Ocultar detalles internos sin necesitar clases ni un framework
Adapter	api.js traduce el contrato OpenAPI del Anexo B a funciones JavaScript simples (obtenerTiposCalzado(), generarCotizacion())	Si el contrato HTTP cambia, solo se ajusta este archivo; el resto de la app no lo nota
Factory simple (función constructora)	Una función construirRequestCotizacion(estado) arma el body de POST /api/cotizaciones a partir del estado actual	Evita construir ese objeto en más de un lugar si mañana se agrega otro punto de entrada (por ejemplo, un botón “cotizar de nuevo”)
Observer ligero (callback de suscripción)	state.js expone una función onChange(callback) que app.js usa para volver a pintar cuando el estado cambia	Evita que app.js tenga que acordarse de llamar a render() manualmente después de cada acción

Con estos tres steering files listos, el siguiente paso es crear el propio Feature Spec del frontend y cargar en él la especificación funcional de pantallas — ver sección 3.6.7.

- Si backend y frontend viven en el mismo repositorio, estos tres steering files se guardan igual en .kiro/steering/ junto a los del backend, distinguibles por su nombre (architecture.md quedaría duplicado; conviene nombrarlos frontend-architecture.md, frontend-conventions.md y frontend-design-patterns.md). Si viven en repositorios separados, se crean tal cual en el workspace del frontend. En nuestro caso los repositorios son separados, distintos para el backend y frontend.

3.6.7 Crear el Feature Spec del frontend y cargar la especificación funcional de pantallas

Este Feature Spec (por ejemplo, cotizador-frontend) es donde entra la especificación de la sección 3.3.4 — pantallas, componentes, estados y reglas de interacción — que Kiro usará para diseñar la interfaz. A diferencia del Feature Spec del backend, aquí la fuente de verdad funcional no son las reglas de negocio (RN-01 a RN-05, ya resueltas del lado del servidor), sino el comportamiento de la pantalla y el contrato de API que consume.

1. Abrir el panel de Specs de Kiro en el workspace del frontend y crear un nuevo Feature Spec. Elegir Requirements-First, igual que en el backend (sección 3.6.3).
2. En el cuadro de descripción inicial, pegar el contenido completo de la sección 3.3.4 (componentes, estados de pantalla y reglas de interacción UI-01 a UI-04), y agregar el contrato de API del Anexo B como referencia de los datos que la pantalla debe consumir y enviar.

Descripción inicial sugerida para el Feature Spec del frontend (pegar la sección 3.3.4 y el contrato del Anexo B antes de esta instrucción):

Con base en la especificación de pantallas y el contrato de API anteriores, genera el requirements.md de un frontend web sin framework (HTML, CSS y JavaScript nativo) para el cotizador de reparación de calzado. Usa notación EARS para cada criterio de aceptación de interfaz. La pantalla es única: no agregues rutas, navegación ni pantallas adicionales no descritas arriba. No implementes ninguna regla de cálculo: el total y el tiempo estimado siempre vienen de la respuesta de POST /api/cotizaciones.

3. Kiro genera requirements.md. Compararlo contra los cinco estados de pantalla (UI-E1 a UI-E5) y las cuatro reglas de interacción (UI-01 a UI-04): cada uno debe reconocerse en algún criterio de aceptación EARS. Prestar especial atención a que Kiro no haya inventado reglas de cálculo del lado del cliente — esa es la

desviación más común en este paso, porque el modelo “ve” los montos del ejemplo Gherkin y puede intentar reproducir la fórmula en JavaScript en lugar de simplemente mostrar lo que devuelve la API.

4. Aprobar requirements.md solo cuando refleje fielmente la sección 3.3.4, y continuar con design.md y tasks.md siguiendo exactamente los mismos criterios de revisión de las secciones 3.6.4 y 3.6.5, pero validando design.md contra frontend-architecture.md y frontend-design-patterns.md en lugar de los steering files del backend.

Qué carga cada Feature Spec, en una frase

El Feature Spec del backend (3.6.3) responde “¿cuánto cuesta y cuánto tarda?” a partir de reglas de negocio. El Feature Spec del frontend (3.6.7) responde “¿qué ve y qué puede hacer el cliente en la pantalla?” a partir de componentes y estados de interfaz. Ninguno de los dos debe intentar responder la pregunta del otro.

3.6.8 Ejecutar las tareas y validar contra la especificación

Con tasks.md aprobado, ejecutar las tareas una por una — nunca con la opción de “ejecutar todas las tareas” de una vez, incluso si Kiro la ofrece: el valor pedagógico de esta sección está en revisar cada pieza generada contra la spec, no en la velocidad. En la sesión activa de Kiro dar la siguiente instrucción:

Ejecuta las tareas en orden una por una. Pide mi aprobación antes de ejecutar cada tarea

1. Verificar la ejecución de cada tarea en el archivo tasks.md.
2. Cuando Kiro termine, verifique los archivos creados y/o la lógica agregada.
3. Si Kiro pide permiso para instalar dependencias o ejecutar comandos, leer qué va a instalar antes de aceptar — igual que en la Sección 1, aceptar sin leer reproduce el riesgo del vibe coding dentro de un flujo que se suponía disciplinado.
4. En este laboratorio no ejecute las tareas relacionadas a pruebas ya que puede consumir mas tokens y hacerle falta para culminar el taller.

3.6.9 Estructura del proyecto backend

Stack sugerido: Java 17 + Spring Boot. Esta es la estructura que design.md y tasks.md deberían converger a producir; sirve como referencia para validar el trabajo de Kiro, no como algo que se copia manualmente.

```
cotizador-backend/
.kiro/
  steering/
    architecture.md
    conventions.md
    design-patterns.md
  specs/
    requirements.md
    design.md
    tasks.md
cotizador-backend/
src/main/java/com/tallerdae/cotizador/
  domain/
    model/
      Calzado.java
      TipoReparacion.java
      Cotizacion.java
      Dinero.java
      NivelUrgencia.java
```

```

exception/
  SinReparacionesSeleccionadasException.java
application/
  port/in/
    GenerarCotizacionUseCase.java
    ConsultarCatalogoUseCase.java
  port/out/
    CotizacionRepositoryPort.java
    CalzadoRepositoryPort.java
    ReparacionRepositoryPort.java
  service/
    GenerarCotizacionService.java
    ConsultarCatalogoService.java
  strategy/
    UrgencyPricingStrategy.java
    RecargoUrgentePricingStrategy.java
infrastructure/
  adapter/in/rest/
    CotizacionController.java
    CatalogoController.java
    dto/CotizacionRequest.java
    dto/CotizacionResponse.java
    mapper/CotizacionMapper.java
  adapter/out/persistence/
    InMemoryCotizacionRepositoryAdapter.java
    InMemoryCalzadoRepositoryAdapter.java
    InMemoryReparacionRepositoryAdapter.java
  config/
    CorsConfig.java
src/test/java/com/tallerdae/cotizador/
  domain/CotizacionTest.java
  application/GenerarCotizacionServiceTest.java

```

3.6.10 Estructura del proyecto frontend

Stack sugerido: HTML + CSS + JavaScript sin framework (deliberadamente simple, consume la API vía fetch). Los tres steering files quedan junto al proyecto, además de state.js como módulo explícito de estado (sección 3.6.6.1):

```

cotizador-frontend/
  .kiro/
    steering/
      architecture.md
      conventions.md
      design-patterns.md
    specs/
      requirements.md
      design.md
      tasks.md
  cotizador-frontend/
    index.html
    css/
      estilos.css
    js/
      state.js      (estado en memoria: catálogo, selección actual, última cotización)
      api.js        (llamadas fetch a /api/tipos-calzado, /api/tipos-reparacion, /api/cotizaciones)
      app.js        (escucha eventos del DOM, coordina state.js y api.js, decide qué renderizar)
    README.md

```

3.7 Paso 5 — Opcional - Validación automatizada contra la especificación

El backend generado debe pasar, como mínimo, pruebas automatizadas equivalentes a los tres escenarios Gherkin del Anexo C. Ejemplo de una de ellas en JUnit 5:

```
@Test
void cotizacionSimpleSinUrgencia_calculaTotalCorrecto() {
    Calzado zapatoFormal = new Calzado("1", "Zapato formal", 1.2);
    TipoReparacion cambioTacon = new TipoReparacion(
        "1", "Cambio de tacón", new Dinero(12.00, "USD"), 2);

    Cotizacion cotizacion = generarCotizacionService.generar(
        zapatoFormal, List.of(cambioTacon), NivelUrgencia.NORMAL);

    assertEquals(14.40, cotizacion.getTotal().getMonto());
    assertEquals(2, cotizacion.getTiempoEstimadoDias());
}
```

Checklist de validación final antes de dar por cerrado el ejercicio:

- Las pruebas automatizadas de los tres escenarios del Anexo C pasan (verde).
- El backend expone exactamente los tres endpoints del Anexo B, ni más ni menos.
- El frontend consume esos endpoints y muestra el total y el tiempo estimado de entrega.
- La estructura de carpetas del backend respeta las tres capas hexagonales sin dependencias invertidas (domain no importa nada de infrastructure).
- Los nombres de clases, métodos y paquetes generados coinciden con la tabla de convenciones (3.4.2).

3.8 Entregables de la sección

- Carpeta seccion-2-spec-driven-back/cotizador-backend/ con el proyecto backend funcional, sus pruebas y la carpeta .kiro/ completa (steering/ y specs/ con requirements.md, design.md y tasks.md tal como quedaron aprobados en Kiro).
- Carpeta seccion-2-spec-driven-front/cotizador-frontend/ con el proyecto frontend funcional, sus pruebas y la carpeta .kiro/ completa (steering/ y specs/ con requirements.md, design.md y tasks.md tal como quedaron aprobados en Kiro).
- Capturas de las tres fases de aprobación dentro de Kiro (requirements.md, design.md y tasks.md) para el Feature Spec del backend y frontend.
- Bitácora de los mensajes intercambiados con Kiro durante la ejecución de tareas (Paso 4), en el mismo formato que la Sección 1, para poder comparar ambas experiencias en el cierre.

3.9 Rúbrica de evaluación — Sección 2

Criterio	Insuficiente (0-59)	Aceptable (60-79)	Sobresaliente (80-100)
Compleitud de la especificación	Faltan reglas de negocio o escenarios clave	Cubre lo esencial, con vacíos menores	Spec completa, trazable y sin sobre-especificación

Criterio	Insuficiente (0-59)	Aceptable (60-79)	Sobresaliente (80-100)
Uso del flujo de Specs de Kiro	Se generó código sin pasar por requirements.md/design.md/tasks.md, o se aprobaron sin revisión	Se siguieron las tres fases, con revisión superficial	Cada fase se revisó activamente y se solicitaron ajustes antes de aprobar
Arquitectura hexagonal	Capas mezcladas o dependencias invertidas	Capas presentes, con alguna fuga de dependencia	Separación estricta domain / application / infrastructure
Convenciones y patrones	Nomenclatura inconsistente, patrones ausentes	Convenciones mayormente respetadas, 1-2 patrones aplicados	Todas las convenciones y los patrones correctamente aplicados

4. Sección 3 — Despliegue integrado con Vibe Coding y pruebas end-to-end

4.1 Objetivo de la sección

Cerrar el círculo entre las dos secciones anteriores: tomar el ambiente Docker desplegado con vibe coding en la Sección 1 (MySQL + Nginx) y extenderlo, otra vez con Kiro en modo conversacional, para alojar los proyectos de backend y frontend construidos con spec-driven development en la Sección 2, de modo que ambos corran integrados detrás de un único punto de entrada. Luego, validar el sistema completo con pruebas end-to-end ejecutadas desde el navegador, siguiendo los escenarios Gherkin de negocio (Anexo C) y de interfaz (sección 3.3.4).

Por qué esta sección vuelve a ser vibe coding, no spec-driven

Conectar tres piezas que ya existen (un ambiente Docker, un backend y un frontend) es exactamente el tipo de tarea que la matriz de decisión de la sesión teórica clasifica como apta para vibe coding: es infraestructura de desarrollo/pruebas, de bajo riesgo, reversible en segundos, y no introduce ninguna regla de negocio nueva que proteger con una especificación formal. Las reglas de negocio y las reglas de interfaz ya fueron fijadas con disciplina en la Sección 2; aquí solo se trata de cablear piezas ya validadas, así que pedirle a Kiro una nueva ronda de requirements.md/design.md/tasks.md sería sobre-especificar un problema de plomería, no de diseño.

4.2 Contexto y alcance

Al igual que en la Sección 1, aquí no se espera una especificación previa: se detona el resultado describiéndole a Kiro la intención completa en lenguaje natural y se itera conversacionalmente hasta que el sistema integrado funciona. La diferencia frente a la Sección 1 es que ahora Kiro no parte de una carpeta vacía, sino de tres artefactos que ya existen y que no deben tocarse en su lógica interna: el docker-compose.yml de la Sección 1, y los proyectos cotizador-backend/ y cotizador-frontend/ de la Sección 2.

Límite del ejercicio

Vibe coding aquí significa libertad para decidir cómo se conectan los contenedores (Dockerfile, variables de entorno, configuración de Nginx), no libertad para modificar las reglas de negocio, los nombres de clases o la arquitectura hexagonal acordados en la Sección 2. Si al integrar aparece la tentación de “solo ajustar rápido” una regla de negocio para que algo cuadre, esa señal indica que el problema no es de despliegue: hay que volver a la Sección 2 y corregirlo ahí, con el gate correspondiente, no aquí a golpe de prompt.

4.3 Herramientas requeridas

- Kiro en modo conversacional / chat (igual que en la Sección 1; no se usa el modo Specs de la Sección 2).
- El ambiente Docker de la Sección 1 (servicios db y web) funcionando o fácilmente reproducible con docker compose up -d.
- El proyecto cotizador-backend de la Sección 2 compilando localmente (mvn package o equivalente).
- El proyecto cotizador-frontend de la Sección 2 funcionando localmente al abrir index.html.
- Un navegador con herramientas de desarrollador (DevTools) para las pruebas de la sección 4.6.

4.4 Arquitectura objetivo del despliegue integrado

El resultado esperado de la conversación con Kiro debe aproximarse al siguiente conjunto de servicios, extendiendo los dos que ya existían desde la Sección 1:

Servicio	Origen	Rol en el despliegue integrado
db	Sección 1 (sin cambios)	Base de datos relacional, disponible para el bonus de persistencia (Paso 5)
backend	Nuevo — empaqueta cotizador-backend/	Expone la API interna en el puerto 8080, solo dentro de la red de Docker
web	Sección 1 (se reconfigura)	Sirve cotizador-frontend/ como contenido estático y hace de proxy hacia /api/ en el backend

El backend no se publica directamente al host: el único punto de entrada visible desde fuera del ambiente sigue siendo Nginx, en el puerto 8080, igual que en la Sección 1.

4.5 Guía paso a paso

Paso 1 — Prompt de intención inicial

Abrir Kiro en modo chat sobre la carpeta raíz que contiene los tres proyectos (el docker-compose.yml de la Sección 1, cotizador-backend/ y cotizador-frontend/) y describir la intención completa en una sola conversación, sin detallar sintaxis de Dockerfile ni de Nginx.

Prompt sugerido (adaptar libremente):

Tengo un docker-compose.yml con MySQL y Nginx en la carpeta seccion-1-vibe-coding, y dos proyectos ya contruidos: proyecto java cotizador-backend (Spring Boot 3.3.5 con Java 21 y packaging JAR cotizador-0.0.1-SNAPSHOT.jar, expone su API en el puerto 8080) en la carpeta seccion-2-spec-driven-back y cotizador-frontend (HTML/CSS/JS sin framework) en la carpeta seccion-2-spec-driven-front. Quiero que trabajes en la carpeta seccion-1-vibe-coding y extiendas el docker-compose.yml para que el backend corra como un contenedor más, sin publicar su puerto directamente al host, y que Nginx sirva los archivos estáticos de cotizador-frontend y actúe como proxy hacia el backend para todo lo que empiece con /api/. Todo debe seguir levantándose con un solo comando.

Paso 2 — Generación del Dockerfile y del servicio backend

Dejar que Kiro proponga el Dockerfile del backend (build en dos etapas: compilar con Maven, correr sobre un runtime de Java liviano) y el nuevo bloque backend: dentro del docker-compose.yml. Revisar el resultado reconstruyendo el ambiente, sin todavía leer línea por línea cada archivo generado — coherente con la naturaleza del vibe coding.

Paso 3 — Opcional - Ajustes conversacionales

Iterar en lenguaje natural sobre el resultado inicial. Ejemplos de instrucciones de seguimiento que se recomienda practicar:

- “El backend no debe publicar su puerto al host, solo debe ser accesible desde dentro de la red de Docker.”
- “Haz que Nginx sirva los archivos de cotizador-frontend como contenido estático y agrega un location /api/ que reenvíe al backend.”

- “Agrega un `depends_on` para que Nginx espere a que el backend haya arrancado antes de aceptar tráfico.”
- “Revisa si `api.js` del frontend está usando una URL absoluta o una ruta relativa; si es absoluta, ajústala para que funcione detrás de Nginx sin cambiar de origen.”

Por qué esta última instrucción importa

Si `api.js` quedó escrito apuntando a rutas relativas (`/api/tipos-calzado`, `/api/cotizaciones`), el frontend no necesita ningún cambio para funcionar detrás de Nginx: el navegador simplemente pide esas rutas al mismo origen (`localhost:8080`) que sirvió la página, y Nginx las reenvía al backend. Si quedó una URL absoluta con host y puerto, es el momento de pedirle a Kiro que la corrija — pero como ajuste de configuración, no como una reapertura de la especificación de la Sección 2.

Paso 4 — Levantar y validar el ambiente integrado

Comandos de validación:

```
docker compose up -d --build
docker compose ps
curl -I http://localhost:8080/
curl -s http://localhost:8080/api/tipos-calzado
```

Confirmar tres cosas antes de pasar al navegador: la base de datos sigue healthy; el backend responde a través de Nginx (no directamente — el puerto 8081 no debe ser alcanzable desde el host); y `http://localhost:8080/` muestra el formulario del cotizador, no la página de bienvenida de Nginx de la Sección 1.

Paso 5 — Iteración adicional (bonus)

Con el mismo estilo conversacional, pedir a Kiro al menos una mejora adicional a elección:

- Conectar el backend a MySQL en lugar del repositorio en memoria: la especificación técnica de la Sección 2 acepta explícitamente memoria (sección 3.2), así que esto es un extra, no un requisito. Es un buen ejercicio para comprobar el beneficio real de la arquitectura hexagonal: se agrega una nueva clase `CotizacionJpaRepositoryAdapter` que implementa el mismo puerto `CotizacionRepositoryPort`, sin tocar el dominio ni los casos de uso.
- Agregar un healthcheck al backend y hacer que Nginx espere a que esté healthy, no solo iniciado.
- Pedirle a Kiro un `README.md` que documente cómo levantar y apagar el ambiente integrado completo.

4.6 Pruebas integrales desde el navegador

Con el sistema completo corriendo detrás de `http://localhost:8080`, cada escenario Gherkin de negocio y de interfaz se convierte en un caso de prueba manual ejecutado directamente en el navegador, usando la interfaz real en lugar de llamadas sueltas a la API. A diferencia de los pasos anteriores, este es el único punto de la sección donde no se le pide nada más a Kiro: es el estudiante, no la IA, quien verifica que el resultado funciona de verdad.

4.6.1 Guion de pruebas manuales basado en los escenarios Gherkin

Ejecutar cada fila en orden sobre `http://localhost:8080`, con las herramientas de desarrollador del navegador abiertas en la pestaña Network para confirmar que cada petición sale hacia `/api/...` (Nginx), no hacia un puerto distinto.

#	Escenario (Anexo C / 3.3.4)	Resultado esperado en el navegador
1	Cotización simple sin urgencia	Total mostrado: 14.40. Tiempo de entrega: 2 días.
2	Cotización urgente con recargo	Subtotal 30.00, total 39.00, tiempo de entrega: 2 días.
3	Cotización con múltiples reparaciones	Total mostrado: 18.00. Tiempo de entrega: 3 días.
4	Botón deshabilitado sin selección completa	El botón “Cotizar” aparece deshabilitado hasta marcar calzado y al menos una reparación.
5	Mostrar resultado tras cotizar exitosamente	El panel de resultado aparece con subtotal, total y tiempo estimado tras pulsar “Cotizar”.
6	Ocultar el resultado anterior al cambiar la selección	Al cambiar el tipo de calzado después de ver un resultado, el panel de resultado desaparece.
7	Mostrar error cuando el servidor rechaza la solicitud	Al forzar un envío sin reparaciones (por ejemplo, con DevTools), se muestra el mensaje de error de la API sin perder la selección.

Registrar el resultado real (pasa / falla) de cada fila; este guion completado es uno de los entregables de la sección (4.8).

4.6.2 Opcional (bonus): automatizar las pruebas end-to-end con Playwright

Para quienes quieran ir un paso más allá del guion manual, Playwright permite automatizar el mismo recorrido del navegador. No sustituye el guion manual de 4.6.1 para efectos de entrega, pero es una buena introducción a pruebas E2E automatizadas.

```
npm init -y
npm install -D @playwright/test
npx playwright install chromium
```

e2e/cotizador.spec.ts (ejemplo del escenario 1):

```
import { test, expect } from '@playwright/test';

test('cotización simple sin urgencia calcula el total correcto', async ({ page }) => {
  await page.goto('http://localhost:8080');
  await page.selectOption('#tipo-calzado-select', { label: 'Zapato formal' });
  await page.check('#reparacion-cambio-tacon');
  await page.click('#boton-cotizar');

  await expect(page.locator('#resultado-cotizacion')).toContainText('14.40');
  await expect(page.locator('#resultado-cotizacion')).toContainText('2 días');
});
```

Ejecutar:

```
npx playwright test
```

4.7 Banco de prompts sugeridos

Prompts adicionales que los estudiantes pueden usar como inspiración si se quedan sin ideas para iterar; no es necesario usarlos todos.

1. “Explicame en dos frases qué cambia en este docker-compose.yml respecto al de la Sección 1.”

2. “Simula que Nginx arranca antes que el backend y el usuario ve un error 502; dime cómo lo solucionarías.”
3. “¿El puerto del backend quedó accesible desde mi máquina? Ayúdame a verificarlo y, si es así, ciérralo.”
4. “Sugiere qué cosas de este despliegue integrado NO deberían usarse tal cual en producción.”

4.8 Entregables de la sección

- Carpeta seccion-3-despliegue-integrado/ con el docker-compose.yml extendido, el Dockerfile del backend y la configuración de Nginx actualizada (ver Anexo F como referencia orientativa, no como plantilla a copiar).
- Captura o log de docker compose ps mostrando los servicios (db, web, backend y, si se hizo el bonus, la conexión a MySQL) en estado saludable.
- Captura del navegador mostrando al menos una cotización exitosa servida end-to-end a través de `http://localhost:8080`.
- Bitácora breve (mín. 10 líneas) de la conversación con Kiro: qué se pidió, qué se ajustó y por qué — mismo formato que la Sección 1.
- El guion de pruebas manuales de la sección 4.6.1, completado con el resultado real de cada uno de los siete casos.
- (Opcional) El script de Playwright de 4.6.2 y la salida de su ejecución.

4.9 Preguntas de reflexión

1. ¿Cuántas iteraciones conversacionales fueron necesarias para que los tres servicios funcionaran juntos? ¿Qué fue lo primero que falló?
2. ¿En algún momento tuviste la tentación de ajustar código de negocio o de interfaz de la Sección 2 para que el despliegue “cuadrara”? Si fue así, ¿cómo lo resolviste sin saltarte el gate de la Sección 2?
3. Compara esta sección con la Sección 1: ambas son vibe coding, pero una parte de una carpeta vacía y la otra integra piezas ya construidas con disciplina. ¿Cambió tu forma de iterar con Kiro por eso?
4. Si este despliegue integrado tuviera que promoverse a un ambiente compartido por todo un equipo de trabajo, ¿qué de lo hecho en esta sección seguiría siendo válido y qué necesitaría revisarse con más estructura?

4.10 Rúbrica de evaluación — Sección 3

Criterio	Insuficiente (0-59)	Aceptable (60-79)	Sobresaliente (80-100)
Integración de servicios	Backend y frontend no llegan a comunicarse entre sí	Funcionan juntos con ajustes manuales fuera del docker-compose.yml	Los tres (o cuatro) servicios levantan con un solo <code>docker compose up -d --build</code>
Enrutamiento y exposición	El backend queda accesible directamente desde el host, sin pasar por Nginx	La mayoría del tráfico pasa por <code>/api/</code> , con alguna excepción	El 100% del tráfico hacia el backend pasa por el proxy de Nginx; el puerto del backend no es accesible desde el host

Criterio	Insuficiente (0-59)	Aceptable (60-79)	Sobresaliente (80-100)
Documentación del proceso	Sin bitácora de prompts	Bitácora breve y genérica	Bitácora clara que muestra iteración real y ajustes concretos
Cobertura de pruebas manuales	Faltan varios de los siete escenarios del guion	Se ejecutaron la mayoría, con algún resultado sin registrar	Los siete escenarios de 4.6.1 se ejecutaron y quedaron registrados con su resultado real
Reflexión crítica	Respuestas superficiales	Respuestas correctas pero generales	Conecta explícitamente con los límites del vibe coding y el gate de la Sección 2

5. Cierre del taller: comparación de experiencia

5.1 Tabla de reflexión comparativa

Completar individualmente al finalizar las tres secciones, con base en la bitácora de prompts y el tiempo real invertido.

Dimensión	Sección 1 — Vibe Coding	Sección 2 — Spec-Driven Dev.
Tiempo total invertido	_____	_____
N.º de iteraciones conversacionales con la IA	_____	_____
¿Revisaste todo el código generado?	_____	_____
Nivel de confianza en el resultado (1-5)	_____	_____
¿Podrías explicar cada decisión de diseño?	_____	_____
¿Otra persona podría retomar tu trabajo sin ti?	_____	_____

5.2 Discusión final

1. ¿En qué momento de la Sección 2 sentiste que la especificación te “frenaba”? ¿Ese freno era injustificado o evitó un error?
2. Si tuvieras que mantener este cotizador en producción durante tres años con un equipo rotativo, ¿qué artefactos de la Sección 2 agradecerías tener?
3. Piensa en tu proyecto de curso o en tu trabajo actual: identifica una tarea de esta semana que debería tratarse como vibe coding y otra que debería tratarse como spec-driven development. Justifica ambas con la matriz de decisión de la sesión teórica.
4. La Sección 3 mezcló un ambiente creado con vibe coding (Sección 1) con proyectos creados con spec-driven development (Sección 2). ¿Ese punto de integración debería, a su vez, tratarse con más o con menos estructura que las piezas que conecta? Justifica tu respuesta.

Anexo A — docker-compose.yml de referencia

Referencia orientativa para el docente o para el estudiante que se quede sin avanzar en la Sección 1. No debe entregarse como solución “copiada”: el valor del ejercicio está en la conversación con Kiro, no en este archivo.

```
services:
  db:
    image: mysql:8.0
    container_name: tallerdae-db
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: tallerdae
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - tallerdae-net

  web:
    image: nginx:alpine
    container_name: tallerdae-web
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
      - ./nginx/html:/usr/share/nginx/html:ro
    depends_on:
      db:
        condition: service_healthy
    networks:
      - tallerdae-net

volumes:
  db_data:

networks:
  tallerdae-net:
    driver: bridge
```

Anexo B — Contrato OpenAPI completo

```

openapi: 3.0.3
info:
  title: Cotizador de Reparación de Calzados
  version: "1.0.0"
paths:
  /api/tipos-calzado:
    get:
      summary: Lista los tipos de calzado disponibles
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/TipoCalzado' }
  /api/tipos-reparacion:
    get:
      summary: Lista los tipos de reparación disponibles
      responses:
        '200':
          description: OK
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/TipoReparacion' }
  /api/cotizaciones:
    post:
      summary: Genera una cotización estimada
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/CotizacionRequest' }
      responses:
        '201':
          description: Cotización generada
          content:
            application/json:
              schema: { $ref: '#/components/schemas/CotizacionResponse' }
        '400':
          description: Solicitud inválida
components:
  schemas:
    TipoCalzado:
      type: object
      properties:
        id: { type: string }
        nombre: { type: string }
        factorComplejidad: { type: number, format: float }
    TipoReparacion:
      type: object
      properties:
        id: { type: string }
        nombre: { type: string }
        precioBase: { type: number, format: float }
        tiempoEstimadoDias: { type: integer }
    CotizacionRequest:
      type: object
      required: [tipoCalzadoId, tipoReparacionIds]

```

```
properties:
  tipoCalzadoId: { type: string }
  tipoReparacionIds:
    type: array
    minItems: 1
    items: { type: string }
  urgente: { type: boolean, default: false }
CotizacionResponse:
  type: object
  properties:
    id: { type: string }
    subtotal: { type: number, format: float }
    recargoUrgencia: { type: number, format: float }
    total: { type: number, format: float }
    moneda: { type: string, example: USD }
    tiempoEstimadoDias: { type: integer }
    fechaCreacion: { type: string, format: date-time }
```

Anexo C — Escenarios Gherkin completos

Característica: Generar cotización de reparación de calzado

Escenario: Cotización simple sin urgencia

Dado un calzado "Zapato formal" con factor de complejidad 1.2
Y una reparación "Cambio de tacón" con precio base 12.00 y tiempo estimado 2 días
Cuando el cliente solicita una cotización sin marcar el servicio como urgente
Entonces el total de la cotización debe ser 14.40
Y el tiempo estimado de entrega debe ser 2 días

Escenario: Cotización urgente con recargo

Dado un calzado "Bota de cuero" con factor de complejidad 1.5
Y una reparación "Cambio de suela" con precio base 20.00 y tiempo estimado 4 días
Cuando el cliente solicita una cotización marcando el servicio como urgente
Entonces el subtotal debe ser 30.00
Y el total debe ser 39.00
Y el tiempo estimado de entrega debe ser 2 días

Escenario: Cotización con múltiples reparaciones

Dado un calzado "Zapatilla deportiva" con factor de complejidad 1.0
Y una reparación "Cosido de costura" con precio base 8.00 y tiempo estimado 1 día
Y una reparación "Limpieza y tinturado" con precio base 10.00 y tiempo estimado 3 días
Cuando el cliente solicita una cotización sin marcar el servicio como urgente
Entonces el total de la cotización debe ser 18.00
Y el tiempo estimado de entrega debe ser 3 días

Escenario: Cotización sin reparaciones seleccionadas

Cuando el cliente solicita una cotización sin seleccionar ninguna reparación
Entonces el sistema debe rechazar la solicitud con un error de validación

Escenarios de interfaz (frontend), correspondientes a la sección 3.3.4:

Característica: Interacción de la pantalla del cotizador

Escenario: Botón deshabilitado sin selección completa

Dado que el catálogo de calzados y reparaciones ya se cargó
Y el cliente no ha seleccionado ninguna reparación
Entonces el botón "Cotizar" debe estar deshabilitado

Escenario: Mostrar resultado tras cotizar exitosamente

Dado que el cliente seleccionó un calzado y al menos una reparación
Cuando el cliente presiona "Cotizar"
Entonces el panel de resultado debe mostrar el subtotal, el total
y el tiempo estimado de entrega devueltos por la API

Escenario: Ocultar el resultado anterior al cambiar la selección

Dado que la pantalla ya muestra un resultado de una cotización previa
Cuando el cliente cambia el tipo de calzado seleccionado
Entonces el panel de resultado debe ocultarse hasta la siguiente cotización

Escenario: Mostrar error cuando el servidor rechaza la solicitud

Dado que el cliente presiona "Cotizar" sin haber seleccionado reparaciones
Cuando la API responde con un error 400
Entonces la pantalla debe mostrar el mensaje de error recibido
Y la selección previa del cliente debe permanecer visible

Anexo D — Fragmentos de código de referencia

Fragmentos ilustrativos del dominio y de un puerto, para orientar el nivel de abstracción esperado. No constituyen la solución completa.

Dominio — Cotizacion.java (fragmento):

```
public final class Cotizacion {
    private final String id;
    private final Calzado calzado;
    private final List<TipoReparacion> reparaciones;
    private final NivelUrgencia urgencia;
    private final Dinero total;
    private final int tiempoEstimadoDias;
    private final LocalDateTime fechaCreacion;

    private Cotizacion(String id, Calzado calzado, List<TipoReparacion> reparaciones,
        NivelUrgencia urgencia, Dinero total, int tiempoEstimadoDias) {
        this.id = id;
        this.calzado = calzado;
        this.reparaciones = List.copyOf(reparaciones);
        this.urgencia = urgencia;
        this.total = total;
        this.tiempoEstimadoDias = tiempoEstimadoDias;
        this.fechaCreacion = LocalDateTime.now();
    }

    public static Cotizacion crear(Calzado calzado, List<TipoReparacion> reparaciones,
        NivelUrgencia urgencia, Dinero total, int tiempoEstimadoDias) {
        if (reparaciones == null || reparaciones.isEmpty()) {
            throw new SinReparacionesSeleccionadasException();
        }
        return new Cotizacion(UUID.randomUUID().toString(), calzado, reparaciones,
            urgencia, total, tiempoEstimadoDias);
    }
    // getters omitidos por brevedad
}
```

Puerto de entrada — GenerarCotizacionUseCase.java:

```
public interface GenerarCotizacionUseCase {
    Cotizacion generar(String tipoCalzadoId, List<String> tipoReparacionIds, boolean urgente);
}
```

Anexo E — Glosario

- **Puerto:** interfaz que define un contrato de comunicación entre el dominio/aplicación y el mundo exterior, sin depender de una tecnología concreta.
- **Adaptador:** implementación concreta de un puerto (por ejemplo, un controlador REST o un repositorio JPA).
- **Agregado raíz:** entidad que actúa como punto de entrada consistente a un conjunto de objetos relacionados (en este caso, Cotizacion).
- **Objeto de valor (Value Object):** objeto inmutable definido por sus atributos, sin identidad propia (en este caso, Dinero).
- **Gate de revisión:** punto de control explícito en el que un humano aprueba un artefacto antes de continuar el flujo.
- **Spec drift:** desalineación progresiva entre la especificación y el código cuando los cambios no se canalizan a través de la especificación.

Anexo F — Artefactos de referencia para el despliegue integrado (Sección 3)

Referencia orientativa para la Sección 3. Igual que el Anexo A, no debe entregarse como solución “copiada” sin adaptarla a los nombres reales que cada estudiante haya usado en su backend y frontend.

docker-compose.yml extendido (los cuatro servicios integrados):

```
services:
  db:
    image: mysql:8.0
    container_name: tallerdae-db
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: tallerdae
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - tallerdae-net

  backend:
    build:
      context: ./cotizador-backend
      dockerfile: Dockerfile
    container_name: tallerdae-backend
    restart: unless-stopped
    environment:
      SERVER_PORT: 8081
    expose:
      - "8081"
    networks:
      - tallerdae-net

  web:
    image: nginx:alpine
    container_name: tallerdae-web
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
      - ./cotizador-frontend:/usr/share/nginx/html:ro
    depends_on:
      db:
        condition: service_healthy
      backend:
        condition: service_started
    networks:
      - tallerdae-net

volumes:
```

```
db_data:

networks:
  tallerdae-net:
    driver: bridge
```

cotizador-backend/Dockerfile:

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

nginx/default.conf actualizado:

```
server {
    listen 80;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://backend:8081/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```